

Architecture Plan: Federated LightRAG

1. Executive Summary

- What We Are Building
- What We Are Achieving

2. Background: How LightRAG Works Today

- 2.1 Core Pipeline
- 2.2 Knowledge Graph Construction
- 2.3 Dual-Level Retrieval
- 2.4 Pluggable Storage
- 2.5 Workspace Isolation
- 2.6 Incremental Updates
- 2.7 What LightRAG Lacks (Gaps We Fill)

3. Architecture

- 3.1 System Overview
- 3.2 Why Each Layer Exists
- 3.3 Data Ingestion Flow
- 3.4 Query Flow with Access Control
- 3.5 CDC / Real-Time Sync Architecture
- 3.6 Knowledge Graph with Provenance
- 3.7 Access Control Model

4. Key Interfaces

- 4.1 Base Connector
- 4.2 Access Control Service
- 4.3 Workspace Manager
- 4.4 Modified LightRAG Core

5. Schema Extraction Per Connector

- PostgreSQL
- MySQL
- MongoDB
- REST API (OpenAPI Spec)
- CSV / XLSX
- GraphQL

6. REST API

- Workspaces
- Data Sources
- File Upload
- Querying
- Schema Explorer
- Knowledge Graph
- Access Control
- CDC Status

7. LightRAG vs Federated LightRAG

8. Implementation Plan

- Phase Sequence
- Phase Details

Architecture Plan: Federated LightRAG

Status: Draft **Date:** April 2026 **Stack:** Python Backend + TypeScript Frontend **Use Case:** Enterprise Knowledge Management **Sync Strategy:** Real-time CDC

1. Executive Summary

What We Are Building

An enterprise-grade RAG platform built on top of [LightRAG](#) (graph-enhanced retrieval-augmented generation) that enables organizations to connect multiple heterogeneous data sources, unify their knowledge into a single queryable knowledge graph per workspace, and govern access with fine-grained role-based permissions.

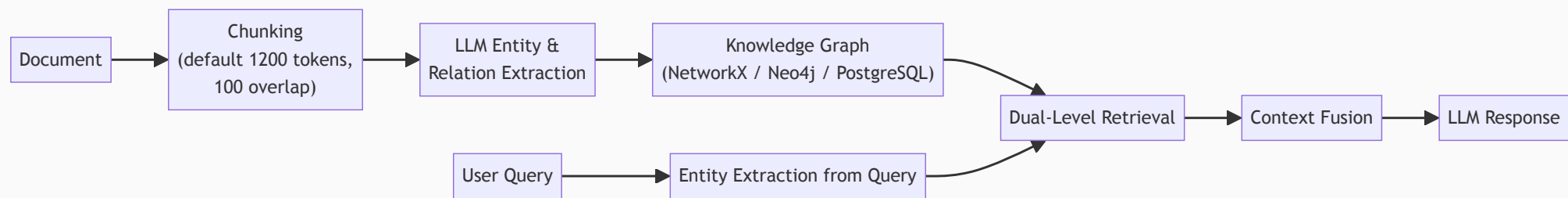
What We Are Achieving

Goal	Outcome
Unified knowledge access	Users ask natural-language questions and get answers drawn from PostgreSQL databases, MongoDB collections, CSV files, PDF documents, REST APIs, and cloud storage — all through a single interface
Data sovereignty	Each workspace is isolated. Users only see data they have permission to access, down to the individual data-source level
Always-current knowledge	Real-time CDC (change data capture) keeps the knowledge graph synchronized with source databases as data changes
Schema visibility	Every connected data source has its information schema extracted, indexed, and explorable — enabling schema-aware queries like “what tables exist in the HR database?”
Enterprise readiness	Multi-workspace isolation, RBAC, OIDC/SAML authentication, audit logging

2. Background: How LightRAG Works Today

LightRAG is a graph-enhanced RAG framework (arXiv:2410.05779, EMNLP 2025). Understanding its architecture is essential because we are extending it, not replacing it.

2.1 Core Pipeline



2.2 Knowledge Graph Construction

What happens: Documents are split into chunks. An LLM extracts entities (Person, Organization, Location, Event, Concept, etc.) and relationships (typed edges with keywords and descriptions) from each chunk.

Entity merging: When two chunks produce entities with the same normalized name, LightRAG merges them — descriptions are concatenated with a `<SEP>` separator, and if the fragment count exceeds `FORCE_LLM_SUMMARY_ON_MERGE` (default: 8), an LLM map-reduce summarization condenses them.

Entity data model (actual graph node properties):

```
{
  "entity_name": "Tesla Inc",
  "entity_type": "organization",
  "description": "American electric vehicle manufacturer<SEP>Also produces solar panels",
  "source_id": "chunk_abc123<SEP>chunk_def456",
  "file_path": "report.pdf<SEP>overview.docx",
  "created_at": 1712345678,
  "weight": 1.0
}
```

2.3 Dual-Level Retrieval

Why two levels: Simple vector search finds isolated facts. Graph traversal finds relationships. Both together give comprehensive answers.

Level	What it retrieves	How	Best for
Low-Level (local)	Specific entities + their direct relationships	Vector search on entity embeddings + 1-2 hop graph traversal	“Who is the CEO of Tesla?”
High-Level (global)	Communities, themes, topics	Community detection on KG, subgraph retrieval	“How does EV adoption affect urban infrastructure?”
Mix (default)	Both levels combined	Both + cross-encoder reranking	General-purpose queries

Six query modes supported: `naive` , `local` , `global` , `hybrid` , `mix` , `bypass` (default: `mix`).

2.4 Pluggable Storage

LightRAG has 4 storage layers, each pluggable:

Storage Layer	Purpose	Available Backends
KV Store	LLM response cache, config	JSON, PostgreSQL, Redis, MongoDB, OpenSearch
Vector Store	Entity/relation embeddings	NanoVectorDB, PostgreSQL (pgvector), Milvus, FAISS, Qdrant, MongoDB, OpenSearch
Graph Store	Knowledge graph nodes/edges	NetworkX (in-memory), Neo4j, PostgreSQL, MongoDB, Memgraph, OpenSearch
Doc Status Store	Document processing status	JSON, PostgreSQL, Redis, MongoDB, OpenSearch

ChromaVectorDBStorage exists but is commented out. AGE (Apache AGE) is also available for PostgreSQL graph storage.

2.5 Workspace Isolation

LightRAG natively supports workspace-based isolation via the `workspace` field:

```
@dataclass
class LightRAG:
    workspace: str = field(default_factory=lambda: os.getenv("WORKSPACE", ""))
    """Workspace for data isolation."""
```

All storage backends use the workspace value as a namespace/prefix, so different workspaces coexist on the same storage backend without data leakage. **We are extending this native concept** with RBAC and federated data sources.

2.6 Incremental Updates

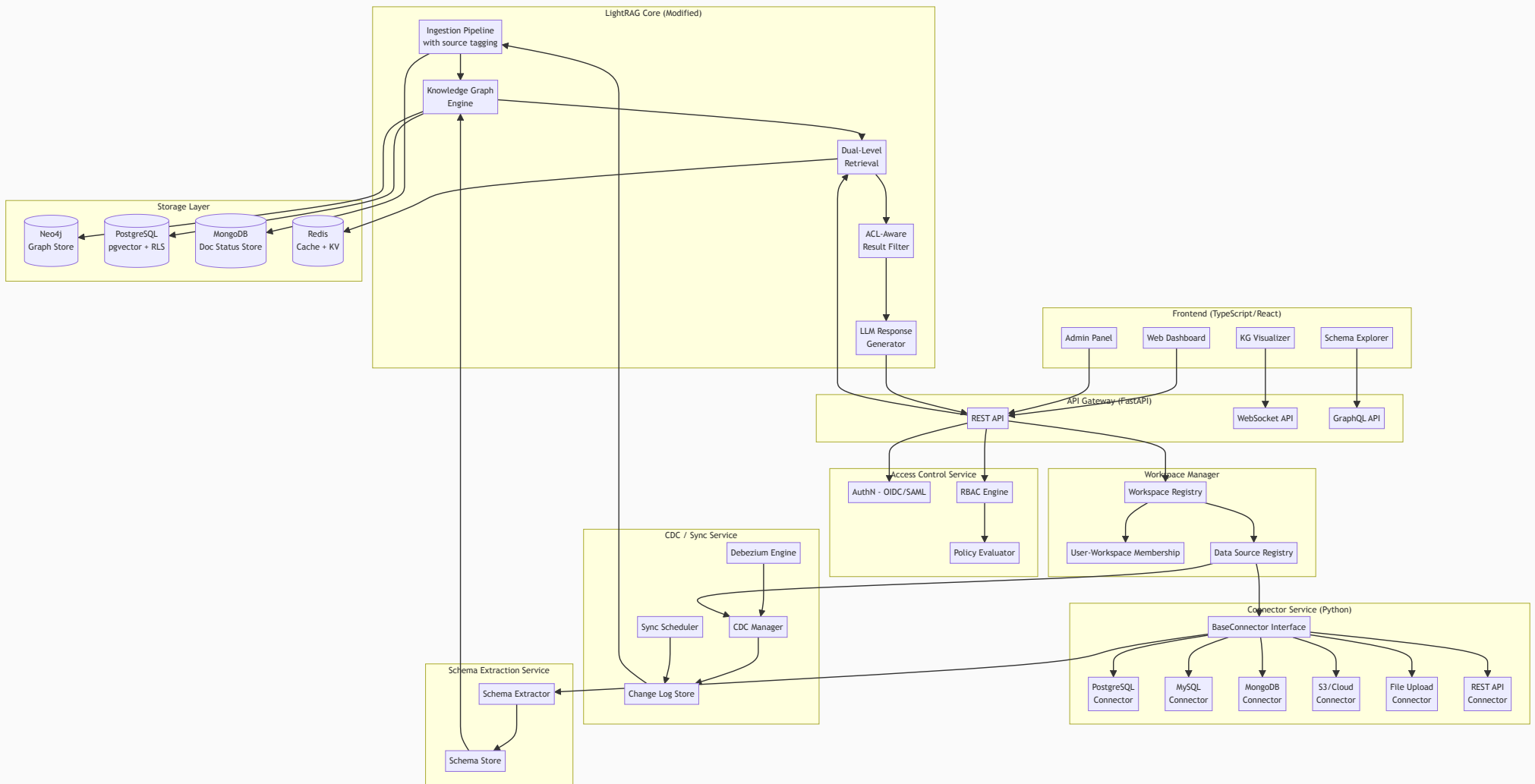
LightRAG supports $O(n)$ incremental updates (vs $O(N)$ full rebuild). New entities are matched by normalized name against the existing graph, merged if found, or created as new nodes. This makes real-time CDC ingestion feasible.

2.7 What LightRAG Lacks (Gaps We Fill)

Gap	Why It Matters	Our Solution
Only accepts file uploads	Enterprise data lives in databases, APIs, cloud storage	Data Source Connector Layer
No access control	Enterprise requires role-based, source-level permissions	RBAC with source-level filtering
No schema awareness	Users need to understand what data exists before querying	Information Schema Extraction
No real-time sync	Data changes constantly; stale KG gives wrong answers	CDC Pipeline (Debezium)
No multi-user workspace management	Teams need isolated workspaces with shared governance	Workspace Manager

3. Architecture

3.1 System Overview



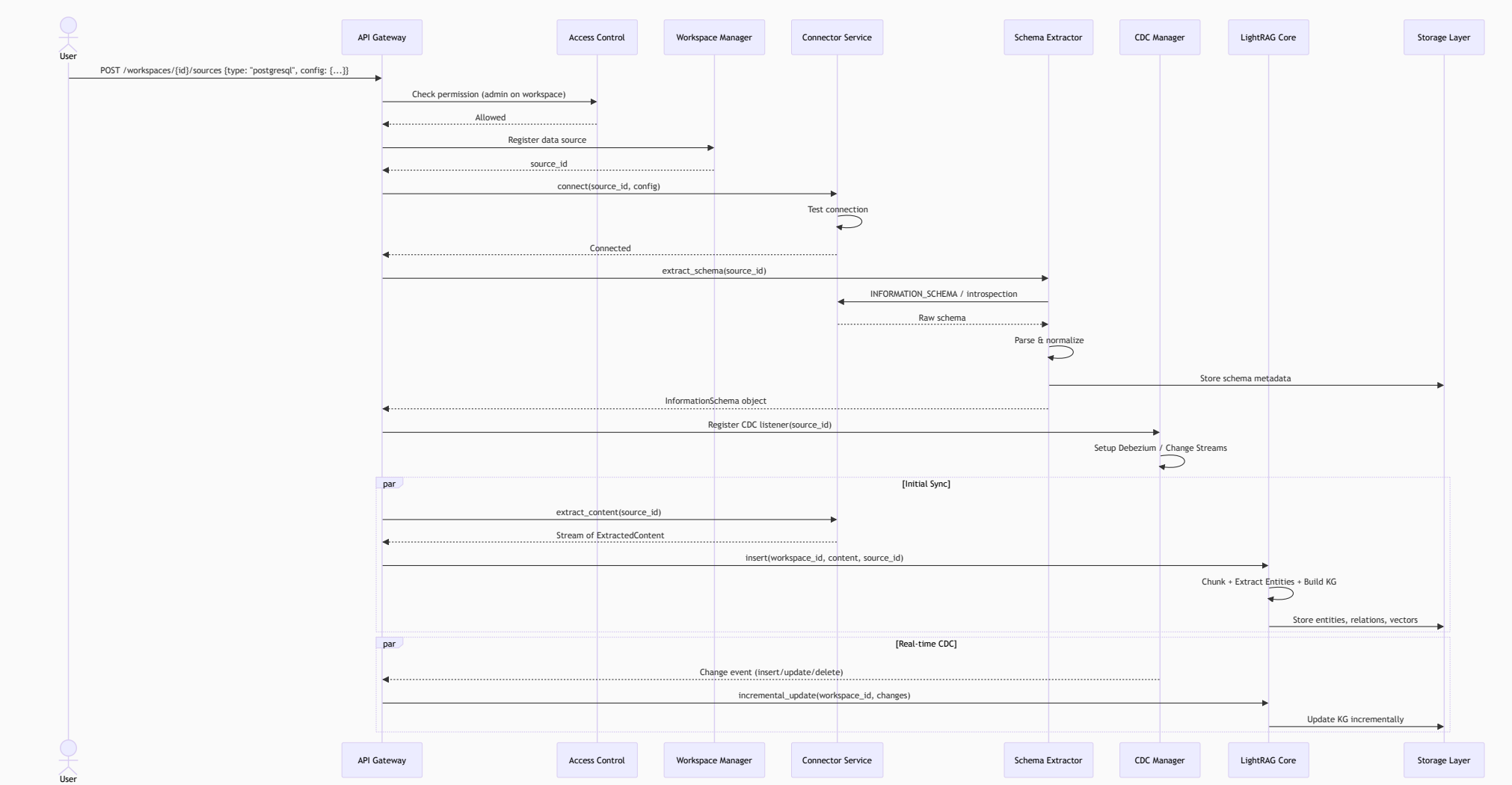
3.2 Why Each Layer Exists

Layer	What It Does	Why We Need It
Frontend	Dashboard, Schema Explorer, KG Visualizer, Admin Panel	Users need a UI to manage workspaces, register data sources, explore schemas, and query the KG visually
API Gateway	REST + WebSocket + GraphQL	REST for CRUD operations, WebSocket for streaming query responses, GraphQL for flexible schema exploration queries

Layer	What It Does	Why We Need It
Access Control	AuthN (OIDC/SAML) + RBAC + Policy evaluation	Enterprise SSO integration; fine-grained permissions per workspace, per data source, per document type
Workspace Manager	Workspace CRUD, membership, data source registry	Extends LightRAG’s native workspace field with user membership and data source tracking
Connector Service	Abstract interface + concrete connectors	Decouples data source specifics from the ingestion pipeline; each connector knows how to connect, extract schema, and stream content
Schema Extraction	Extracts and normalizes information schemas	Users need to discover what data exists before querying; schema nodes are also indexed into the KG for schema-aware retrieval
CDC / Sync Service	Debezium (PostgreSQL WAL, MySQL binlog), MongoDB Change Streams, file watchers	Keeps the knowledge graph current without manual re-ingestion; critical for enterprise where source data changes constantly
LightRAG Core (Modified)	Same dual-level retrieval + source tagging + ACL filtering	Proven retrieval quality; we add provenance metadata (source_id , workspace_id) and filter results at query time
Storage Layer	PostgreSQL (pgvector + RLS), Neo4j, MongoDB, Redis	PostgreSQL for unified storage + row-level security; Neo4j for graph-heavy workloads; Redis for caching; MongoDB as all-in-one option

3.3 Data Ingestion Flow

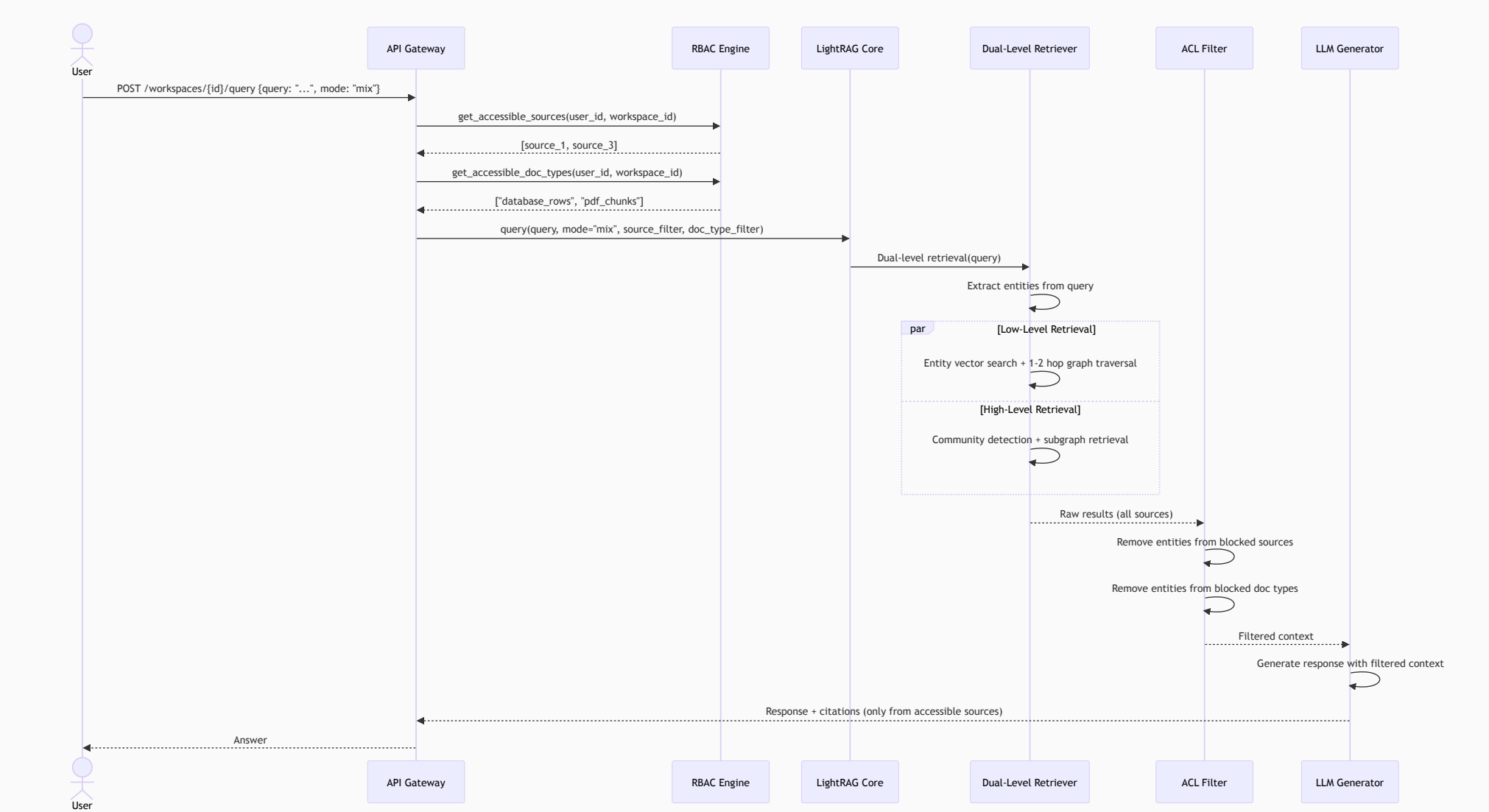
What: When a user registers a data source to a workspace, the system connects to it, extracts its schema, performs an initial content sync, and starts listening for real-time changes.



Why this flow: The initial sync establishes a baseline KG. CDC keeps it current without requiring full re-ingestion. Schema extraction happens first so that schema nodes exist in the KG before content entities reference them.

3.4 Query Flow with Access Control

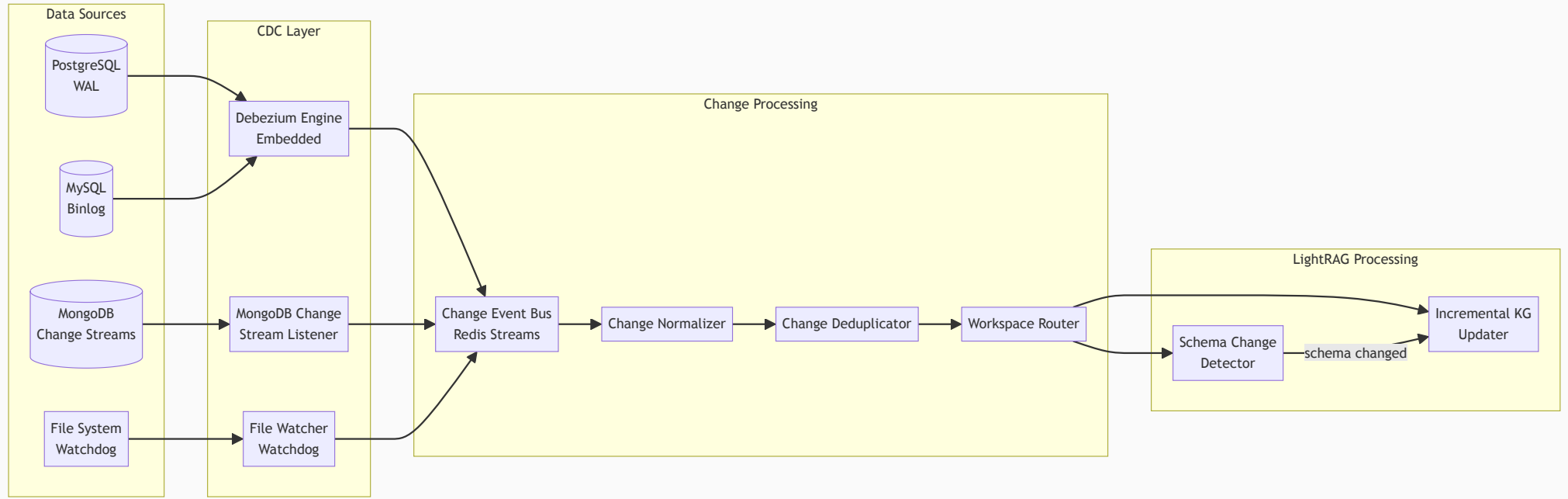
What: When a user queries a workspace, the system retrieves relevant context from the KG, filters out anything the user cannot access, and generates a response.



Why ACL filtering happens post-retrieval: Filtering at query time (rather than at storage time) preserves the KG’s structural integrity — the graph is complete, but users only see slices they’re authorized for. This avoids graph fragmentation and enables admin-level queries across all sources.

3.5 CDC / Real-Time Sync Architecture

What: A change data capture pipeline that monitors source databases and files for changes, then updates the knowledge graph incrementally.

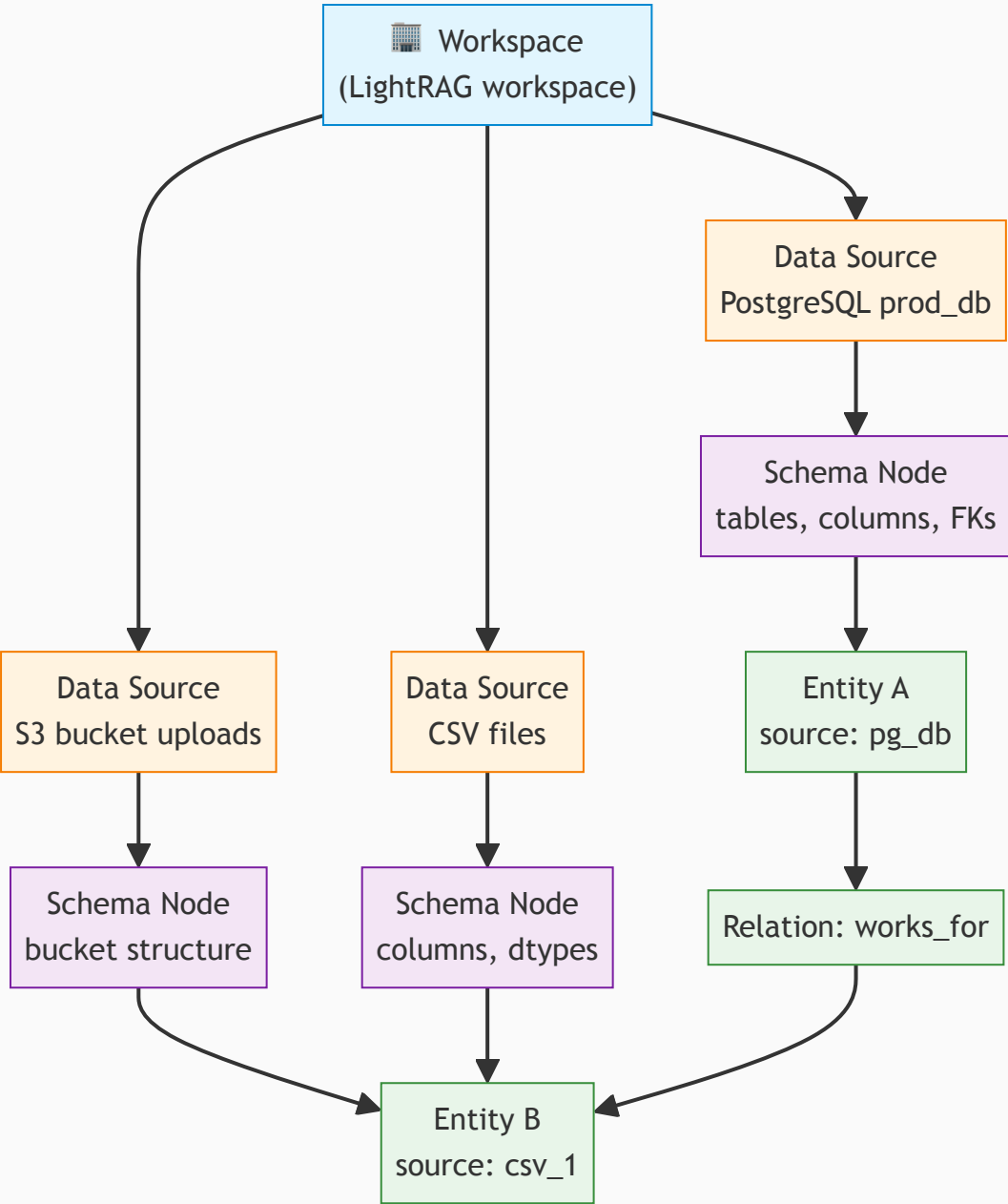


Why Debezium embedded (not Kafka Connect): Running Debezium embedded in the Python process avoids the operational overhead of a separate Kafka/Connect cluster. For enterprise scale, this can be swapped to a standalone Debezium Server.

Why a Change Event Bus: Decouples change detection from KG update. Multiple changes to the same entity are deduplicated before reaching LightRAG, avoiding unnecessary LLM calls for entity summarization.

3.6 Knowledge Graph with Provenance

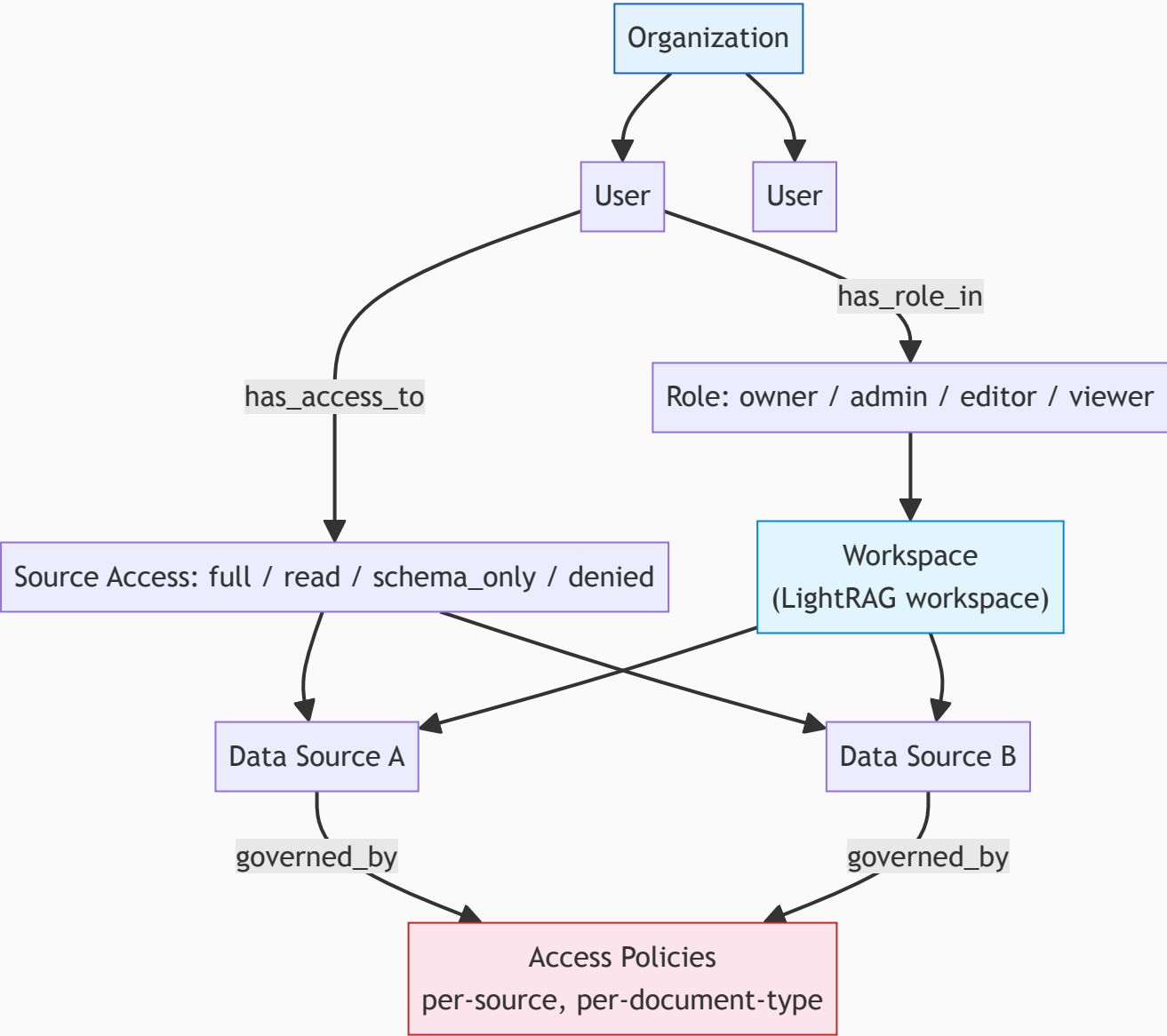
What: The KG is enriched with schema nodes and provenance metadata so every entity traces back to its source.



Why provenance metadata: Every entity, relation, and chunk is tagged with `workspace_id` , `source_id` , `source_type` , and `extraction_timestamp` . This enables: - ACL filtering at query time (filter by `source_id`) - Source deletion (remove all entities from a disconnected source) - Audit trails (trace which source contributed which fact) - Incremental CDC updates (update only entities from changed sources)

3.7 Access Control Model

What: Multi-layer RBAC governing who can access which workspaces, data sources, and document types.



Why source-level access: In an enterprise, different teams own different databases. The HR team should query HR data but not Finance data, even within the same workspace. Source-level access enables this without duplicating the KG.

Access levels:

Level	Can See Schema	Can Query Data	Can Modify
full	Yes	Yes	Yes
read	Yes	Yes	No
schema_only	Yes (metadata only)	No	No
denied	No	No	No

Why `schema_only` : Sometimes a user needs to know that a table called `employees` exists with columns `name` , `salary` , `department` — but should not see actual employee records. This enables data catalog use cases without exposing sensitive data.

Enforcement points:

Point	What Happens	Why
Query time	Retrieved KG entities are filtered by accessible sources and doc types before LLM generation	Most critical; prevents any data leakage in responses
Indexing time	Every entity/relation is tagged with <code>source_id</code> and <code>workspace_id</code>	Enables filtering; provenance for audit
Storage layer	PostgreSQL RLS policies, Neo4j label-based ACL, MongoDB field-level redaction	Defense in depth; prevents direct DB access bypass

4. Key Interfaces

4.1 Base Connector

What: An abstract interface that all data source connectors implement. Each connector knows how to connect to a specific type of source, extract its schema, stream content, and listen for changes.

Why we need it: Without a common interface, each data source type would require bespoke integration code scattered throughout the system. The `BaseConnector` abstraction lets us add new source types (e.g., Elasticsearch, Snowflake) by implementing a single class.

```
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from datetime import datetime
from enum import Enum
from typing import Any, AsyncIterator, Dict, List, Optional

class ConnectorType(str, Enum):
    POSTGRESQL = "postgresql"
    MYSQL = "mysql"
    MONGODB = "mongodb"
    SQLITE = "sqlite"
    SQLSERVER = "sqlserver"
    ORACLE = "oracle"
    S3 = "s3"
    GCS = "gcs"
    AZURE_BLOB = "azure_blob"
    CSV = "csv"
    XLSX = "xlsx"
    PDF = "pdf"
    DOCX = "docx"
    REST_API = "rest_api"
    GRAPHQL = "graphql"
    KAFKA = "kafka"

class SyncMode(str, Enum):
    MANUAL = "manual"
```

```
SCHEDULED = "scheduled"
CDC = "cdc"
```

```
class ChangeType(str, Enum):
    INSERT = "insert"
    UPDATE = "update"
    DELETE = "delete"
    SCHEMA_CHANGE = "schema_change"
```

```
@dataclass
```

```
class ColumnSchema:
    name: str
    data_type: str
    nullable: bool
    default_value: Optional[str]
    is_primary_key: bool
    is_foreign_key: bool
    foreign_key_ref: Optional[str] # "table.column"
    description: Optional[str]
    sample_values: List[Any] = field(default_factory=list)
```

```
@dataclass
```

```
class TableSchema:
    name: str
    schema_name: Optional[str]
    columns: List[ColumnSchema]
    row_count: Optional[int]
    description: Optional[str]
```

```
@dataclass
```

```
class InformationSchema:
    source_id: str
    source_type: ConnectorType
    tables: List[TableSchema]
    raw_schema: Dict[str, Any]
    extracted_at: datetime
    stats: Dict[str, Any]
```

```
@dataclass
```

```
class Document:
    content: str
    metadata: Dict[str, Any]
    source_id: str
    resource_name: str
    doc_type: str # "database_row", "pdf_page", "csv_row", etc.
```

```
@dataclass
```

```
class ExtractedContent:
    source_id: str
    documents: List[Document]
    schema: Optional[InformationSchema] = None
```

```
@dataclass
```

```
class ChangeEvent:
    source_id: str
    workspace_id: str
    change_type: ChangeType
    resource_name: str
    primary_key: Optional[Dict[str, Any]]
    before: Optional[Dict[str, Any]]
    after: Optional[Dict[str, Any]]
    timestamp: datetime
```

```
@dataclass
```

```

class ResourceDescriptor:
    name: str
    type: str # "table", "collection" (MongoDB), "file", "endpoint"
    row_count: Optional[int]
    size_bytes: Optional[int]
    last_modified: Optional[datetime]
    description: Optional[str]

class BaseConnector(ABC):
    connector_type: ConnectorType

    @abstractmethod
    async def connect(self, config: Dict[str, Any]) -> bool:
        """Establish connection to the data source."""

    @abstractmethod
    async def test_connection(self) -> bool:
        """Verify the connection is alive."""

    @abstractmethod
    async def extract_schema(self) -> InformationSchema:
        """Extract the full information schema from this data source."""

    @abstractmethod
    async def extract_content(
        self,
        resource_filter: Optional[List[str]] = None,
        since: Optional[datetime] = None,
    ) -> AsyncIterator[ExtractedContent]:
        """
        Extract content for indexing.
        - resource_filter: specific tables/files/endpoints to target
        - since: incremental extraction (only changed data)
        """

    @abstractmethod
    async def list_resources(self) -> List[ResourceDescriptor]:
        """List all available resources (tables, files, endpoints)."""

    @abstractmethod
    async def get_changes(self, since: datetime) -> List[ChangeEvent]:
        """Detect changes since last sync (for incremental updates)."""

    @abstractmethod
    async def supports_cdc(self) -> bool:
        """Whether this connector supports real-time CDC."""

    @abstractmethod
    async def start_cdc_listener(
        self, callback: callable, since: Optional[datetime] = None
    ) -> None:
        """Start listening for real-time changes."""

    async def stop_cdc_listener(self) -> None:
        """Stop listening for changes."""

    async def disconnect(self) -> None:
        """Clean up connection resources."""

```

4.2 Access Control Service

What: Manages user permissions at the workspace level (role) and data source level (access level).

Why we need it: Enterprise knowledge management requires that different teams see different data. A single RBAC flag is insufficient — we need source-level and document-type-level granularity.

```

from dataclasses import dataclass
from enum import Enum
from typing import Dict, List

```

```

class Role(str, Enum):
    OWNER = "owner"
    ADMIN = "admin"
    EDITOR = "editor"
    VIEWER = "viewer"

class SourceAccessLevel(str, Enum):
    FULL = "full"
    READ = "read"
    SCHEMA_ONLY = "schema_only"
    DENIED = "denied"

@dataclass
class UserPermission:
    user_id: str
    workspace_id: str
    role: Role
    source_access: Dict[str, SourceAccessLevel] # source_id -> access level
    doc_type_access: Dict[str, bool] # doc_type -> allowed

class AccessControlService:
    async def get_accessible_sources(
        self, user_id: str, workspace_id: str
    ) -> List[str]: ...

    async def get_accessible_doc_types(
        self, user_id: str, workspace_id: str
    ) -> List[str]: ...

    async def check_source_access(
        self, user_id: str, source_id: str, required_level: SourceAccessLevel
    ) -> bool: ...

    async def get_user_permissions(
        self, user_id: str, workspace_id: str
    ) -> UserPermission: ...

    async def grant_source_access(
        self, admin_user_id: str, target_user_id: str,
        source_id: str, level: SourceAccessLevel
    ) -> bool: ...

    async def revoke_source_access(
        self, admin_user_id: str, target_user_id: str, source_id: str
    ) -> bool: ...

```

4.3 Workspace Manager

What: Extends LightRAG's native `workspace` field with user membership, data source registration, and per-workspace settings.

Why we need it: LightRAG only uses workspace as a storage namespace. We need workspace as a first-class entity with members, registered data sources, and governance policies.

```

from dataclasses import dataclass
from datetime import datetime
from typing import Any, Dict, List, Optional

@dataclass
class WorkspaceSettings:
    llm_model: str = "gpt-4o"
    embedding_model: str = "text-embedding-3-large"
    chunk_size: int = 1200
    chunk_overlap: int = 100
    default_retrieval_mode: str = "mix"

```

```
enable_schema_aware_retrieval: bool = True
enable_citations: bool = True
```

```
@dataclass
```

```
class Workspace:
    id: str
    name: str
    owner_id: str
    organization_id: str
    description: str
    settings: WorkspaceSettings
    created_at: datetime
    updated_at: datetime
```

```
@dataclass
```

```
class DataSourceRegistration:
    id: str
    workspace_id: str
    connector_type: ConnectorType
    connector_config: Dict[str, Any]  # Encrypted at rest
    display_name: str
    description: Optional[str]
    sync_mode: SyncMode
    sync_interval_seconds: Optional[int]
    last_synced_at: Optional[datetime]
    schema_snapshot: Optional[InformationSchema]
    status: str  # "connected" | "error" | "syncing" | "disconnected"
    error_message: Optional[str]
```

```
class WorkspaceManager:
    async def create_workspace(
        self, name: str, owner_id: str, org_id: str, settings: WorkspaceSettings
    ) -> Workspace: ...

    async def register_data_source(
        self, workspace_id: str, connector_type: ConnectorType,
        config: Dict[str, Any], display_name: str, sync_mode: SyncMode
    ) -> DataSourceRegistration: ...

    async def remove_data_source(self, source_id: str) -> bool: ...

    async def trigger_sync(self, source_id: str) -> str: ...

    async def get_workspace_sources(
        self, workspace_id: str
    ) -> List[DataSourceRegistration]: ...

    async def add_user_to_workspace(
        self, user_id: str, workspace_id: str, role: Role
    ) -> bool: ...

    async def remove_user_from_workspace(
        self, user_id: str, workspace_id: str
    ) -> bool: ...
```

4.4 Modified LightRAG Core

What: LightRAG's core with three additions: source provenance tagging, ACL-aware retrieval, and CDC-driven incremental updates.

Why we need it: Vanilla LightRAG treats all data equally. We need it to know where each entity came from and filter results based on who's asking.

```
from typing import List
```

```
class FederatedLightRAG:
    def __init__(
```

```
self,
workspace_id: str,
storage_config: Dict[str, Any],
llm_config: Dict[str, Any],
embedding_config: Dict[str, Any],
): ...

async def insert_with_provenance(
    self,
    content: ExtractedContent,
    workspace_id: str,
    source_id: str,
) -> None:
    """
    Insert content into the KG with source tagging.
    Every entity/relation/chunk gets:
    - workspace_id
    - source_id
    - source_type
    - extraction_timestamp
    """

async def query_with_acl(
    self,
    query: str,
    mode: str,
    accessible_sources: List[str],
    accessible_doc_types: List[str],
):
    """
    Dual-level retrieval with ACL filtering.
    1. Perform normal dual-level retrieval
    2. Filter results to only include accessible sources/doc_types
    3. Generate response with filtered context
    """

async def incremental_update(
    self,
    changes: List[ChangeEvent],
    workspace_id: str,
) -> None:
    """
    Process CDC change events:
    - INSERT: Extract entities, add to KG
    - UPDATE: Re-extract affected entities, merge in KG
    - DELETE: Remove entities that are no longer present
    - SCHEMA_CHANGE: Re-extract schema, update schema nodes
    """

async def delete_source_data(self, source_id: str) -> None:
    """
    Remove all entities/relations/chunks from a specific source.
    Used when a data source is disconnected from a workspace.
    """

async def get_source_schema(self, source_id: str) -> InformationSchema: ...
```

5. Schema Extraction Per Connector

What: Each connector extracts the information schema (tables, columns, types, foreign keys) from its data source. This schema is stored as metadata nodes in the knowledge graph.

Why we need it: Users need to discover what data exists. Schema-aware queries like “what tables are in the HR database?” or “which columns contain PII?” become possible. Schema metadata also helps the LLM generate better answers when querying structured data.

PostgreSQL

```
SELECT
t.table_schema,
```

```
t.table_name,  
c.column_name,  
c.data_type,  
c.is_nullable,  
c.column_default,  
tc.constraint_type,  
kcu_referenced.table_name AS referenced_table,  
kcu_referenced.column_name AS referenced_column  
FROM information_schema.tables t  
JOIN information_schema.columns c  
  ON t.table_name = c.table_name AND t.table_schema = c.table_schema  
LEFT JOIN information_schema.key_column_usage kcu  
  ON c.table_name = kcu.table_name AND c.column_name = kcu.column_name  
LEFT JOIN information_schema.table_constraints tc  
  ON kcu.constraint_name = tc.constraint_name  
LEFT JOIN information_schema.referential_constraints rc  
  ON tc.constraint_name = rc.constraint_name  
LEFT JOIN information_schema.key_column_usage kcu_referenced  
  ON rc.unique_constraint_name = kcu_referenced.constraint_name  
WHERE t.table_schema NOT IN ('pg_catalog', 'information_schema')  
ORDER BY t.table_schema, t.table_name, c.ordinal_position;
```

MySQL

```
SELECT  
  t.TABLE_SCHEMA,  
  t.TABLE_NAME,  
  c.COLUMN_NAME,  
  c.DATA_TYPE,  
  c.IS_NULLABLE,  
  c.COLUMN_DEFAULT,  
  c.COLUMN_KEY,  
  kcu.REFERENCED_TABLE_NAME,  
  kcu.REFERENCED_COLUMN_NAME  
FROM information_schema.TABLES t  
JOIN information_schema.COLUMNS c  
  ON t.TABLE_SCHEMA = c.TABLE_SCHEMA AND t.TABLE_NAME = c.TABLE_NAME  
LEFT JOIN information_schema.KEY_COLUMN_USAGE kcu  
  ON c.TABLE_SCHEMA = kcu.TABLE_SCHEMA  
  AND c.TABLE_NAME = kcu.TABLE_NAME  
  AND c.COLUMN_NAME = kcu.COLUMN_NAME  
  AND kcu.REFERENCED_TABLE_NAME IS NOT NULL  
WHERE t.TABLE_SCHEMA NOT IN ('mysql', 'information_schema', 'performance_schema', 'sys')  
ORDER BY t.TABLE_SCHEMA, t.TABLE_NAME, c.ORDINAL_POSITION;
```

MongoDB

```
db.collection.aggregate([  
  { $sample: { size: 1000 } },  
  { $project: {  
    typeMap: { $type: "$$ROOT" },  
    fields: { $objectToArray: "$$ROOT" }  
  }},  
  { $unwind: "$fields" },  
  { $group: {  
    _id: "$fields.k",  
    types: { $addToSet: "$fields.v.type" },  
    count: { $sum: 1 }  
  } }  
)
```

REST API (OpenAPI Spec)

```
def extract_openapi_schema(base_url: str) -> dict:
    response = requests.get(f"{base_url}/openapi.json")
    spec = response.json()
    return {
        "paths": list(spec.get("paths", {}).keys()),
        "schemas": spec.get("components", {}).get("schemas", {}),
        "info": spec.get("info", {}),
    }
```

CSV / XLSX

```
def extract_file_schema(file_path: str) -> dict:
    df = pd.read_csv(file_path, nrows=100)
    return {
        "columns": [
            {
                "name": col,
                "dtype": str(df[col].dtype),
                "null_rate": float(df[col].isnull().mean()),
                "unique_count": int(df[col].nunique()),
                "sample_values": df[col].head(5).tolist(),
            }
            for col in df.columns
        ],
        "row_count": len(df),
    }
```

GraphQL

```
def extract_graphql_schema(endpoint: str) -> dict:
    introspection_query = """
    {
        __schema {
            types { name kind fields { name type { name kind } } }
            queryType { name }
            mutationType { name }
        }
    }
    """

    response = requests.post(endpoint, json={"query": introspection_query})
    return response.json()["data"]["__schema"]
```

6. REST API

What: The API surface users and administrators interact with.

Why REST + WebSocket + GraphQL: REST for straightforward CRUD. WebSocket for streaming query responses (LLM tokens arrive incrementally). GraphQL for flexible schema exploration (users can query exactly the schema fields they need).

Workspaces

Method	Endpoint	What It Does
POST	/api/v1/workspaces	Create workspace
GET	/api/v1/workspaces	List user's workspaces
GET	/api/v1/workspaces/{id}	Get workspace details
PUT	/api/v1/workspaces/{id}	Update workspace settings

Method	Endpoint	What It Does
DELETE	/api/v1/workspaces/{id}	Delete workspace

Data Sources

Method	Endpoint	What It Does
POST	/api/v1/workspaces/{id}/sources	Register data source
GET	/api/v1/workspaces/{id}/sources	List data sources
GET	/api/v1/workspaces/{id}/sources/{sid}	Get source details + status
PUT	/api/v1/workspaces/{id}/sources/{sid}	Update source config
DELETE	/api/v1/workspaces/{id}/sources/{sid}	Remove source
POST	/api/v1/workspaces/{id}/sources/{sid}/sync	Trigger manual sync
GET	/api/v1/workspaces/{id}/sources/{sid}/schema	Get information schema

File Upload

Method	Endpoint	What It Does
POST	/api/v1/workspaces/{id}/upload	Upload files to workspace

Querying

Method	Endpoint	What It Does
POST	/api/v1/workspaces/{id}/query	Query the workspace’s KG
GET	/api/v1/workspaces/{id}/query/modes	List available query modes
POST	/api/v1/workspaces/{id}/query/stream	Stream query response

Schema Explorer

Method	Endpoint	What It Does
GET	/api/v1/workspaces/{id}/schema	Get all schemas (federated)
GET	/api/v1/workspaces/{id}/schema/{sid}	Get specific source schema
GET	/api/v1/workspaces/{id}/schema/search?q=	Search across schemas

Knowledge Graph

Method	Endpoint	What It Does
GET	/api/v1/workspaces/{id}/graph/stats	KG statistics
GET	/api/v1/workspaces/{id}/graph/entities	List entities

Method	Endpoint	What It Does
GET	/api/v1/workspaces/{id}/graph/relations	List relations
GET	/api/v1/workspaces/{id}/graph/visualize	KG visualization data

Access Control

Method	Endpoint	What It Does
POST	/api/v1/workspaces/{id}/members	Add member
GET	/api/v1/workspaces/{id}/members	List members
PUT	/api/v1/workspaces/{id}/members/{uid}	Update role
DELETE	/api/v1/workspaces/{id}/members/{uid}	Remove member
PUT	/api/v1/workspaces/{id}/sources/{sid}/access	Set source-level access

CDC Status

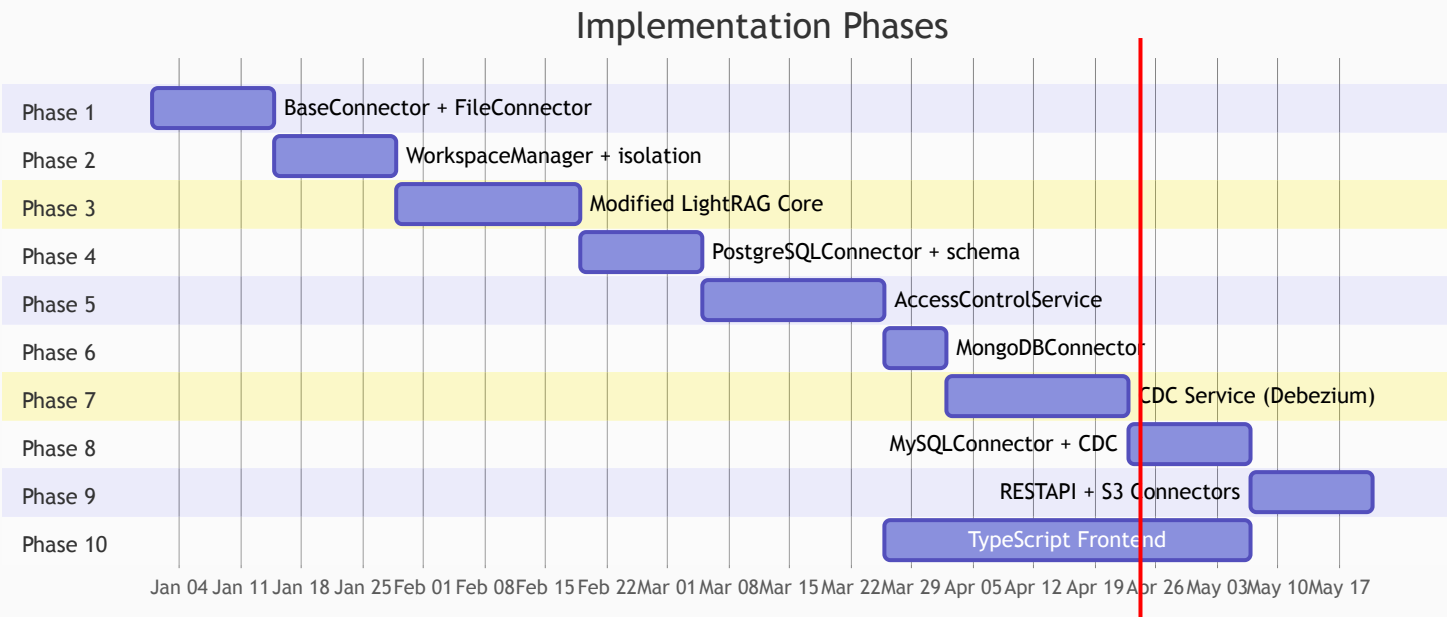
Method	Endpoint	What It Does
GET	/api/v1/workspaces/{id}/cdc/status	CDC pipeline status
GET	/api/v1/workspaces/{id}/cdc/changelog	Recent changes

7. LightRAG vs Federated LightRAG

Component	LightRAG (Current)	Federated LightRAG (Proposed)	What We Achieve
Data ingestion	File upload only	File + DB connectors + API + Cloud storage	Query any enterprise data source
Storage isolation	Native workspace (namespace)	Workspace + RBAC + source-level ACL	Multi-team governance
Access control	None	RBAC with source + doc-type granularity	Data sovereignty
Schema awareness	None	Per-source schema extraction in KG	Data catalog + schema-aware answers
Retrieval	Graph + Vector	Graph + Vector + source-filtered + schema-aware	Accurate, governed answers
Entity provenance	Not tracked	source_id , workspace_id per entity	Audit trail + source deletion
Sync	Manual insert	Manual + Scheduled + Real-time CDC	Always-current knowledge
Default chunk size	1200 tokens	Same (configurable per workspace)	No regression

8. Implementation Plan

Phase Sequence



Phase Details

Phase	What	Why This Order	Effort
1	BaseConnector + FileConnector	Foundation first; FileConnector is simplest and validates the abstraction	1-2 weeks
2	WorkspaceManager + storage isolation	Must exist before we can register sources into workspaces	1-2 weeks
3	Modified LightRAG Core	Source tagging + provenance in KG; needed before any connector can insert data	2-3 weeks
4	PostgreSQLConnector + schema extraction	First database connector; validates schema extraction + ingestion end-to-end	1-2 weeks
5	AccessControlService	RBAC engine + query-time filtering; security is needed before multi-user access	2-3 weeks
6	MongoDBConnector	Second database connector; validates the abstraction works for NoSQL	1 week
7	CDC Service (Debezium)	Real-time sync for PostgreSQL; most complex operational component	2-3 weeks
8	MySQLConnector + CDC	MySQL support; reuses CDC patterns from Phase 7	1-2 weeks
9	RESTAPIConnector + S3Connector	API + cloud storage; broader enterprise coverage	2 weeks
10	TypeScript Frontend	Dashboard, Schema Explorer, KG Visualizer; can start in parallel with Phase 5	4-6 weeks

Total: 17-28 weeks (4-7 months)

9. References

- **LightRAG Paper:** Guo, Z., Xia, L., Yu, Y., Ao, T., & Huang, C. (2024). *LightRAG: Simple and Fast Retrieval-Augmented Generation*. arXiv:2410.05779. EMNLP 2025.

- **LightRAG GitHub:** <https://github.com/HKUDS/LightRAG>
- **LightRAG Source:** `lightrag/lightrag.py` , `lightrag/operate.py` , `lightrag/kg/__init__.py` , `lightrag/constants.py` , `lightrag/base.py`
- **Federated Database Systems:** Sheth, A.P. & Larson, J.A. (1990). *Federated Database Systems for Managing Distributed, Heterogeneous, and Autonomous Databases*. ACM Computing Surveys.
- **Debezium (CDC):** <https://debezium.io/>
- **NodeRAG:** Xu, T. et al. (2025). *NodeRAG: Structuring Graph-based RAG with Heterogeneous Nodes*. arXiv:2504.11544.
- **KG-Infused RAG:** Wu, D. et al. (2025). *KG-Infused RAG: Augmenting Corpus-Based RAG with External Knowledge Graphs*. arXiv:2506.09542.